

使用 Agent Development Kit (ADK) 和 GenAI Eval Service 建構及評估 BigQuery 代理

來源：<https://codelabs.developers.google.com/bigquery-adk-eval?hl=zh-tw>

步驟數：14

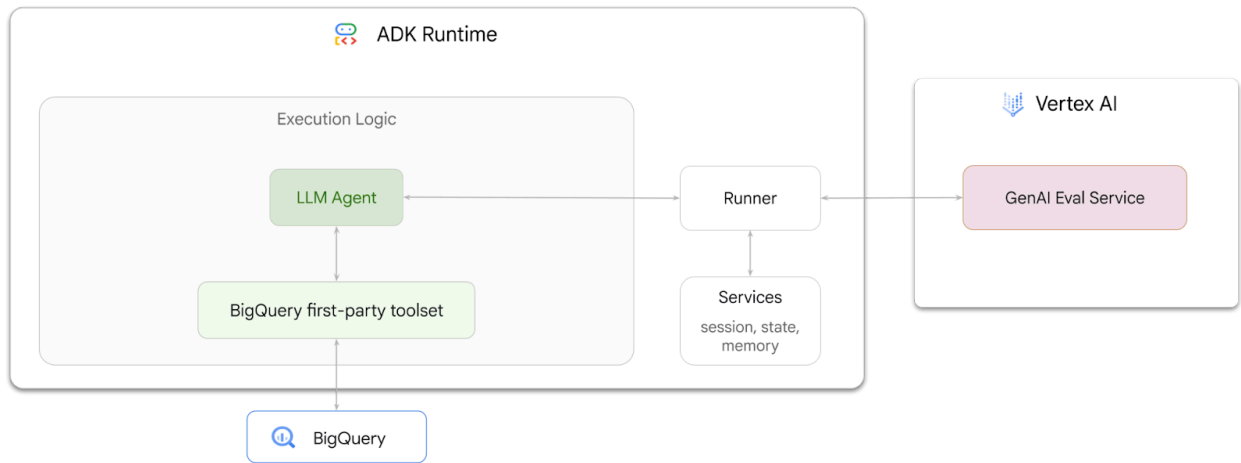
使用 ADK 和 BigQuery 工具組，建構可與 BigQuery 資料互動的代理程式。您也會瞭解如何使用 Vertex AI GenAI Eval 服務評估虛擬服務專員。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 建立 BigQuery 資料集](#)
4. [4. 準備 ADK 代理程式環境](#)
5. [5. 建立 ADK 應用程式](#)
6. [6. 定義代理並指派 BigQuery 工具組](#)
7. [7. 準備評估代理程式](#)
8. [8. 建立評估資料集](#)
9. [9. 定義評估指標](#)
10. [10. 建立評估工作](#)
11. [11. 執行評估作業](#)
12. [12. 提升代理的評估結果](#)
13. [13. 清除](#)
14. [14. 恭喜](#)

1. 簡介

在本程式碼實驗室中，您將瞭解如何使用 **Agent Development Kit (ADK)**，建構可回答 **BigQuery** 中所儲存資料相關問題的代理。您也會使用 **Vertex AI 的 GenAI Evaluation Service** 評估這些代理：



執行步驟

- 在 **ADK** 中建構對話式數據分析代理
- 為這個代理程式配備 ADK 的 **BigQuery 第一方工具集**，以便與儲存在 BigQuery 中的資料互動
- 使用 **Vertex AI 生成式 AI 評估服務** 為代理建立評估架構
- 針對一組黃金回覆，對這個代理執行評估

軟硬體需求

- 網路瀏覽器，例如 **Chrome**
- 已啟用計費功能的 Google Cloud 專案，或
- Gmail 帳戶。下一節將說明如何兌換這個程式碼研究室的 \$5 美元免費抵免額，並設定新專案

本程式碼研究室適合各種程度的開發人員 (包括初學者)。您將使用 Google Cloud Shell 中的指令列介面和 Python 程式碼進行 ADK 開發。您不需要是 Python 專家，但瞭解如何解讀程式碼有助於理解概念。

步驟 2 · 約 5 分鐘

2. 事前準備

info

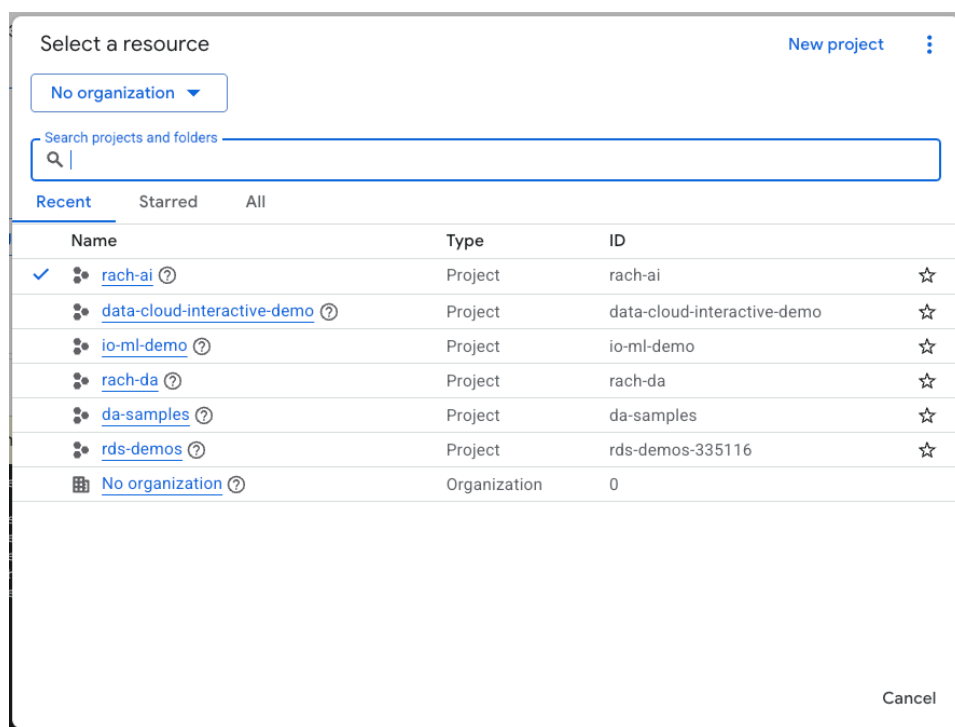
從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。

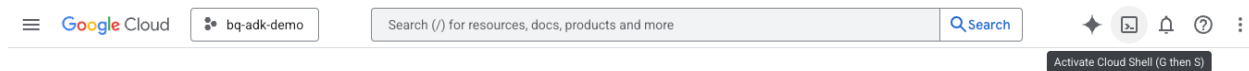


2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」：



2. 連線至 Cloud Shell 後，請執行下列指令，在 Cloud Shell 中驗證您的身分：

```
gcloud auth list
```

3. 執行下列指令，確認專案已設定為搭配 gcloud 使用：

```
gcloud config list project
```

4. 使用下列指令設定專案：

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

啟用 API

1. 執行下列指令，啟用所有必要 API 和服務：

```
gcloud services enable bigquery.googleapis.com \
    aiplatform.googleapis.com \
    cloudresourcemanager.googleapis.com
```

2. 成功執行指令後，您應該會看到類似下方的訊息：

```
Operation "operations/..." finished successfully.
```

步驟 3 · 約 5 分鐘

3. 建立 BigQuery 資料集

1. 在 Cloud Shell 中執行下列指令，在 BigQuery 中建立名為「ecommerce」的新資料集：

```
bq mk --dataset --location=US ecommerce
```

BigQuery 公開資料集 [thelook_ecommerce](#) 的靜態子集會以 AVRO 檔案的形式，儲存在公開的 Google Cloud Storage bucket 中。

2. 在 Cloud Shell 中執行這項指令，將這些 Avro 檔案載入 BigQuery 做為資料表 (events、order_items、products、users、orders)：

```
bq load --source_format=AVRO --autodetect \  
ecommerce.events \  
gs://sample-data-and-media/thelook_dataset_snapshot/events/*.avro.gz  
  
bq load --source_format=AVRO --autodetect \  
ecommerce.order_items \  
gs://sample-data-and-media/thelook_dataset_snapshot/order_items/*.avro.gz  
  
bq load --source_format=AVRO --autodetect \  
ecommerce.products \  
gs://sample-data-and-media/thelook_dataset_snapshot/products/*.avro.gz  
  
bq load --source_format=AVRO --autodetect \  
ecommerce.users \  
gs://sample-data-and-media/thelook_dataset_snapshot/users/*.avro.gz  
  
bq load --source_format=AVRO --autodetect \  
ecommerce.orders \  
gs://sample-data-and-media/thelook_dataset_snapshot/orders/*.avro.gz  
bq load --source_format=AVRO --autodetect \  
ecommerce.inventory_items \  
gs://sample-data-and-media/thelook_dataset_snapshot/inventory_items/*.avro.gz  
bq load --source_format=AVRO --autodetect \  
ecommerce.distribution_centers \  
gs://sample-data-and-media/thelook_dataset_snapshot/distribution_centers/*.avro.gz
```

這項程序可能需要幾分鐘才能完成。

3. 前往 Google Cloud 專案中的 **BigQuery 控制台**，確認資料集和資料表已建立：

Explorer

+ Add data

<

Search BigQuery resources

?

Show starred only

▼ bq-adk-demo

► Repositories

► Queries

► Notebooks

► Data canvases

► Data preparations

► Pipelines

► External connections

► ecommerce

☆

⋮

ecommerce

Create table

Sharing

Copy

Delete

Refresh

Data set info

Edit details

Data set ID

bq-adk-demo.ecommerce

Created

5 Sept 2025, 10:52:20 UTC+1

Default table expiry

60 days

Last modified

5 Sept 2025, 10:52:20 UTC+1

Data location

US

Description

Default collation

Default rounding mode

ROUNDING_MODE_UNSPECIFIED

Time travel window

7 days

Case insensitive

false

Labels

Tags



步驟 4 · 約 5 分鐘

4. 準備 ADK 代理程式環境

返回 Cloud Shell，確認您位於主目錄。我們會建立虛擬 Python 環境，並安裝必要套件。

1. 在 Cloud Shell 中開啟新的終端機分頁，然後執行下列指令，建立並前往名為 **bigquery-adk-codelab** 的資料夾：

```
mkdir bigquery-adk-codelab  
cd bigquery-adk-codelab
```

2. 建立 Python 虛擬環境：

```
python -m venv .venv
```

3. 啟動虛擬環境：

```
source .venv/bin/activate
```

4. 安裝 Google 的 ADK 和 AI Platform Python 套件。評估 BigQuery 代理時，需要 AI 平台和 pandas 套件：

```
pip install google-adk google-cloud-aiplatform[evaluation] pandas
```

步驟 5 · 約 5 分鐘

5. 建立 ADK 應用程式

接著建立 BigQuery 代理程式。這個代理程式的設計宗旨，是回答有關 BigQuery 儲存資料的自然語言問題。

1. 執行 **ADK 建立公用程式指令**，使用必要的資料夾和檔案，架構新的代理程式應用程式：

```
adk create data_agent_app
```

按照提示操作：

1. 選擇 `gemini-2.5-flash` 做為模型。
2. 選擇 Vertex AI 做為後端。
3. 確認預設的 Google Cloud 專案 ID 和區域。

以下是互動範例：

```
(.venv) rds@cloudshell:~/bigquery-adk-codelab (bq-adk-demo)$ adk create data_agent_app
Choose a model for the root agent:
1. gemini-2.5-flash
2. Other models (fill later)
Choose model (1, 2): 1
1. Google AI
2. Vertex AI
Choose a backend (1, 2): 2

You need an existing Google Cloud account and project, check out this link for details:
https://google.github.io/adk-docs/get-started/quickstart/#gemini---google-cloud-vertex-ai

Enter Google Cloud project ID [bq-adk-demo]:
Enter Google Cloud region [us-centrall1]:

Agent created in /home/rds/bigquery-adk-codelab/data_agent_app:
- .env
- __init__.py
- agent.py
```

2. 按一下 Cloud Shell 中的「開啟編輯器」按鈕，開啟 Cloud Shell 編輯器並查看新建立的資料夾和檔案：



請注意產生的檔案：

```
bigquery-adk-codelab/
├── .venv/
└── data_agent_app/
    ├── __init__.py
    ├── agent.py
    └── .env
```

- **init.py**: 將資料夾標示為 Python 模組。
- **agent.py**: 包含初始代理程式定義。
- **.env**：包含專案的環境變數 (您可能需要依序點選「View」>「Toggle Hidden Files」來查看這個檔案)

從提示更新任何未正確設定的變數：

GOOGLE_GENAI_USE_VERTEXAI=1

GOOGLE_CLOUD_PROJECT=<YOUR_GOOGLE_PROJECT_ID>

GOOGLE_CLOUD_LOCATION=<YOUR_GOOGLE_CLOUD_REGION>

步驟 6 · 約 5 分鐘

6. 定義代理並指派 BigQuery 工具組

如要定義使用 BigQuery 工具集與 BigQuery 互動的 ADK 代理程式，請將 `agent.py` 檔案的現有內容替換為下列程式碼。

您必須將代理程式指令中的專案 ID 更新為實際專案 ID：

```
from google.adk.agents import Agent
from google.adk.tools.bigquery import BigQueryCredentialsConfig, BigQueryToolset
import google.auth
import dotenv

dotenv.load_dotenv()

credentials, _ = google.auth.default()
credentials_config = BigQueryCredentialsConfig(credentials=credentials)
bigquery_toolset = BigQueryToolset(
    credentials_config=credentials_config
)

root_agent = Agent(
    model="gemini-2.5-flash",
    name="bigquery_agent",
    description="Agent that answers questions about BigQuery data by executing SQL queries.",
    instruction=(
        """
        You are a BigQuery data analysis agent.
        You are able to answer questions on data stored in project-id: '<YOUR_PROJECT_ID>' on
the `ecommerce` dataset.
        """
    ),
    tools=[bigquery_toolset]
)

def get_bigquery_agent():
    return root_agent
```

BigQuery 工具集可讓代理程式擷取中繼資料，並對 BigQuery 資料執行 SQL 查詢。如要使用工具集，您必須進行驗證，最常見的選項包括：用於開發作業的應用程式預設憑證 (ADC)、代理程式需要代表特定使用者採取行動時使用的互動式 OAuth，以及用於安全生產環境層級驗證的服務帳戶憑證。

您可以在這裡與代理程式對話，方法是返回 Cloud Shell 並執行下列指令：

```
adk web
```

您應該會看到通知，表示網頁伺服器已啟動：

...

INFO: Started server process [2735]

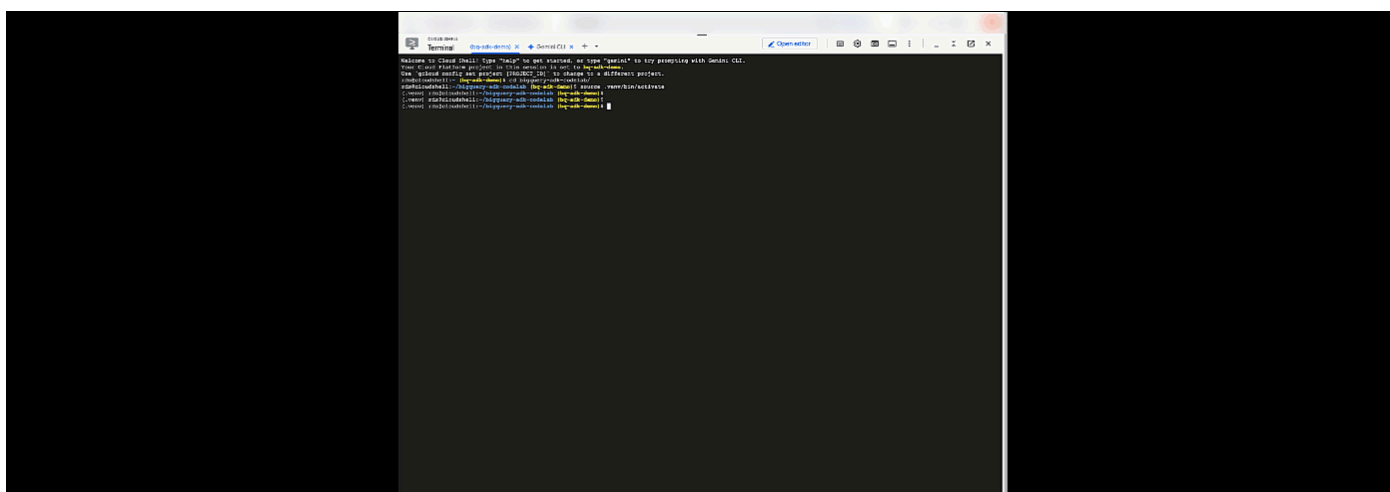
INFO: Waiting for application startup.

```
+-----+
| ADK Web Server started                                     |
|                                                           |
| For local testing, access at http://127.0.0.1:8000.       |
+-----+
```

INFO: Application startup complete.

INFO: Uvicorn running on http://127.0.0.1:8000

按一下提供的網址啟動 ADK 網頁，即可向代理程式詢問資料集相關問題：



關閉 adk web，然後在終端機中按下 Ctrl + C，關閉網路伺服器。

步驟 7 · 約 5 分鐘

7. 準備評估代理程式

您已定義 BigQuery 代理程式，現在需要讓代理程式可供評估。

以下程式碼定義的 `run_conversation` 函式會建立代理、執行工作階段，並處理事件以擷取最終回覆，藉此控管對話流程。

1. 返回 Cloud Editor，在 `bigquery-adk-codelab` 目錄中建立名為 `run_agent.py` 的新檔案，然後複製/貼上下列程式碼：

```

from data_agent_app.agent import get_bigquery_agent
from google.adk.sessions import InMemorySessionService
from google.adk.runners import Runner
from google.genai import types
import uuid

APP_NAME = "data_agent_app"
USER_ID = "bigquery_user_101"

async def run_conversation(prompt: str):
    """Runs a conversation with the BigQuery agent using the ADK Runner."""

    session_service = InMemorySessionService()
    session_id = f"{APP_NAME}-{uuid.uuid4().hex[:8]}"
    root_agent = get_bigquery_agent()

    runner = Runner(
        agent=root_agent, app_name=APP_NAME, session_service=session_service
    )
    session = await session_service.create_session(
        app_name=APP_NAME, user_id=USER_ID, session_id=session_id
    )
    final_response_text = "Unable to retrieve final response."
    tool_calls = []

    try:
        # Run the agent and process the events as they are generated
        async for event in runner.run_async(
            user_id=USER_ID,
            session_id=session_id,
            new_message=types.Content(role="user", parts=[types.Part(text=prompt)]),
        ):

            if (
                event.content
                and event.content.parts
                and event.content.parts[0].function_call
            ):

                func_call = event.content.parts[0].function_call

                tool_call = {
                    "tool_name": func_call.name,
                    "tool_input": dict(func_call.args),
                }
                tool_calls.append(tool_call)

            if event.is_final_response():
                if event.content and event.content.parts:
                    final_response_text = event.content.parts[0].text
                break

```

```
except Exception as e:
    print(f"Error in run_conversation: {e}")
    final_response_text = f"An error occurred during the conversation: {e}"

return {
    "response": final_response_text,
    "predicted_trajectory": tool_calls
}
```

下列程式碼會定義公用程式函式，用於呼叫這個可執行的函式並傳回結果。此外，還包含可列印及儲存評估結果的輔助函式：

2. 在 `bigquery-adk-codelab` 目錄中建立名為 `utils.py` 的新檔案，然後將下列程式碼複製/貼到 `utils.py` 檔案中：

```

import json
import os
import asyncio
import run_agent
import numbers
import math

def get_agent_response(prompt: str) -> dict:
    """Invokes the agent with a prompt and returns its response."""

    try:
        response = asyncio.run(run_agent.run_conversation(prompt)) # Invoke the agent
        return response
    except Exception as e:
        return {"response": "Error: Agent failed to produce a response."}

def save_evaluation_results(eval_result, experiment_run):
    """Processes, saves, and prints the evaluation results for a single run."""

    os.makedirs("eval_results", exist_ok=True)
    output_file_path = os.path.join(
        "eval_results", f"bq_agent_eval_results_{experiment_run}.json"
    )

    # Prepare data for JSON serialization
    eval_result_dict = {
        "summary_metrics": eval_result.summary_metrics,
        "pointwise_metrics": eval_result.metrics_table.to_dict("records"),
    }

    # --- Save the results as a JSON file ---
    with open(output_file_path, "w") as f:
        json.dump(eval_result_dict, f, indent=4)
    print(f"Results for run '{experiment_run}' saved to {output_file_path}")

def print_evaluation_summary(eval_result):
    """Prints a detailed summary of the evaluation results, including summary-level and
    aggregated pointwise metrics."""

    pointwise_metrics = eval_result.metrics_table
    # Print summary metrics for the current run
    summary_metrics = eval_result.summary_metrics
    if summary_metrics:
        for key, value in summary_metrics.items():
            if isinstance(value, numbers.Real) and not math.isnan(value):
                value = f"{value:.2f}"

                metric_name = key.replace("/mean", "").replace("_", " ").title()
                print(f"- {metric_name}: {key}: {value}")
    else:

```



```
    print("No summary metrics found for this run.")
print("\n" + "=" * 50 + "\n")

if not pointwise_metrics.empty:
    total_questions = len(pointwise_metrics)
    avg_completeness_score = pointwise_metrics["completeness_metric/score"].mean()
    avg_factual_accuracy_score = pointwise_metrics[
        "factual_accuracy_metric/score"
    ].mean()
    print("\n" + "=" * 50 + "\n")
    print("--- Aggregated Evaluation Summary ---")
    print(f"Total questions in evaluation dataset: {total_questions}")
    print(f"Average Completeness Score: {avg_completeness_score:.2f}")
    print(f"Average Factual Accuracy Score: {avg_factual_accuracy_score:.2f}")
    print("\n" + "=" * 50 + "\n")
else:
    print("\nNo successful evaluation runs were completed.")
```

步驟 8 · 約 5 分鐘

8. 建立評估資料集

如要評估代理程式，您需要[建立評估資料集](#)、[定義評估指標](#)，以及[執行評估工作](#)。

評估資料集包含問題清單 ([提示](#)) 和對應的正確答案 ([參考資料](#))。評估服務會使用這些配對來比較代理程式的回覆，並判斷回覆是否準確。

1. 在 bigquery-adk-codelab 目錄中建立名為 evaluation_dataset.json 的新檔案，然後複製/貼上下列評估資料集：

```
[
  {
    "prompt": "What tables are available in the dataset `ecommerce`?",
    "reference": "The tables available in the dataset `ecommerce` are:
`distribution_centers`, `events`, `inventory_items`, `order_items`, `orders`, `products`, and
`users`."
  },
  {
    "prompt": "How many users are there in total?",
    "reference": "There are 100,000 users in total."
  },
  {
    "prompt": "Find the email and age of the user with id 72685.",
    "reference": "The email address of user 72685 is lindseybrennan@example.org and their
age is 59."
  },
  {
    "prompt": "How many orders have a status of Complete?",
    "reference": "There are 31,077 orders with a status of 'complete'."
  },
  {
    "prompt": "Which distribution center has the highest latitude, and what is it's
latitude?",
    "reference": "Chicago IL is the distribution center with the highest latitude, with a
latitude of 41.84."
  },
  {
    "prompt": "Retrieve the order id for all orders with a status of cancelled placed on
the 1st June 2023 before 6am.",
    "reference": "The order IDs for all orders with a status of 'cancelled' placed on the
1st June 2023 before 6am are: 26622, 49223"
  },
  {
    "prompt": "What id the full name and user ids of the top 5 users with the most
orders.",
    "reference": "The top 5 users with the most orders are: Kristine Pennington (user ID
77359), Anthony Bright (user ID 4137), David Bean (user ID 30740), Michelle Wright (user ID
54563), and Matthew Reynolds (user ID 41136), each with 4 total orders."
  },
  {
    "prompt": "Which distribution center is associated with the highest average retail
price of its products, and what is the average retail price?",
    "reference": "The distribution center associated with the highest average retail price
of its products is Houston TX, with an average retail price of $69.74."
  },
  {
    "prompt": "How many events were of type 'purchase' in Seoul during May 2024?",
    "reference": "In May 2024, there were 57 'purchase' events recorded in Seoul."
  },
  {
    "prompt": "For orders placed in June 2023, how many took three days or longer to be
delivered after they were shipped?",

```

```

    "reference": "In June 2023, there were 318 orders with a time difference of of 3 days
or more between when they were shipped and delivered."
  },
  {
    "prompt": "What are the names of the products and their respective retail price that
have never been sold, but have a retail price greater than $210?",
    "reference": "The products that have never been sold but have a retail price greater
than $210 are:\n- Tommy Hilfiger Men's 2 Button Side Vent Windowpane Trim Fit Sport Coat,
with a retail price of $249.9\n- MICHAEL Michael Kors Women's Hooded Leather Jacket: $211.11"
  },
  {
    "prompt": "List the id and first name of users between the ages of 70 and 75 who have
Facebook were sourced from Facebook and are located in California.",
    "reference": "The users between the ages of 70 and 75 from California with 'Facebook'
as their traffic source are:\n- Julie (ID: 25379)\n- Sherry (ID: 85196)\n- Kenneth (ID:
82238)\n- Lindsay (ID: 64079)\n- Matthew (ID: 99612)"
  },
  {
    "prompt": "Identify the full name and user id of users over the age of 67 who live
within 3.5 kilometers of any distribution_center.",
    "reference": "The users over the age of 67 who live within 3.5 kilometers of any
distribution center are:\n- William Campbell (user ID: 26082)\n- Becky Cantrell (user ID:
39008)"
  },
  {
    "prompt": "What is the median age of users for each gender?",
    "reference": "The median age for female users is 41, and the median age for male users
is 41."
  },
  {
    "prompt": "What is the average sale price of complete orders compared to returned
orders, and what is the percentage difference (to two decimal places) between them?",
    "reference": "The average sale price for 'Complete' orders was $59.56, while for
'Returned' orders it was $59.76. This represents a percentage difference of 0.34%."
  }
}
]

```

9. 定義評估指標

我們現在會使用兩個自訂指標，評估代理程式回答 BigQuery 資料相關問題的能力，兩者都會提供 1 到 5 分的分數：

- **事實準確度指標**：評估回覆中呈現的所有資料和事實是否準確無誤 (與基準真相相比)。
- **完整度指標**：這項指標會評估回覆是否包含使用者要求的所有重要資訊，以及正確答案中的資訊，且沒有任何重大遺漏。

1. 最後，在 `bigquery-adk-codelab` 目錄中建立名為 `evaluate_agent.py` 的新檔案，然後將指標定義程式碼複製/貼到 `evaluate_agent.py` 檔案中：

```

import uuid
import pandas as pd
from datetime import datetime
from vertexai.preview.evaluation import EvalTask
from vertexai.preview.evaluation.metrics import (
    PointwiseMetricPromptTemplate,
    PointwiseMetric,
    TrajectorySingleToolUse,
)
from utils import save_evaluation_results, print_evaluation_summary, get_agent_response

factual_accuracy_metric = PointwiseMetric(
    metric="factual_accuracy_metric",
    metric_prompt_template=PointwiseMetricPromptTemplate(
        instruction="""You are an expert evaluator assessing the factual accuracy of an AI's
answer to a user's question, given a natural language prompt and a 'reference' (ground truth)
answer. Your task is to determine if all factual information in the AI's answer is precise
and correct when compared to the reference."""",
        criteria={
            "Accuracy": """The AI's answer must present factual information (numerical values,
names, dates, specific values) that are identical to or an exact logical derivation from
the reference.
            - Wording may vary, but the core factual information must be the same.
            - No numerical discrepancies.
            - No incorrect names or identifiers.
            - No fabricated or misleading details.
            - Note: Minor rounding of numerical values that doesn't alter the core meaning or
lead to significant misrepresentation is generally acceptable, assuming the prompt doesn't
ask for exact precision."""",
        },
        rating_rubric={
            "5": "Excellent: The response is entirely factually correct. All factual
information precisely matches the reference. There are absolutely no inaccuracies or
misleading details.",
            "3": "Good: The response is generally accurate, but contains minor, non-critical
factual inaccuracies (e.g., a negligible rounding difference or slightly wrong detail) that
do not impact the core understanding.",
            "1": "Poor: The response contains significant factual errors, major numerical
discrepancies, or fabricated information that makes the answer incorrect or unreliable."
        },
        input_variables=["prompt", "reference", "response"],
    ),
)

completeness_metric = PointwiseMetric(
    metric="completeness_metric",
    metric_prompt_template=PointwiseMetricPromptTemplate(
        instruction="""You are an expert evaluator assessing the completeness of an AI's
answer to a user's question, given a natural language prompt and a 'reference' (ground truth)
answer. Your task is to determine if the AI's answer provides all the essential information
requested by the user and present in the reference."""",
        criteria={

```

```

        "Completeness": ""The AI's answer must include all key pieces of information explicitly or implicitly requested by the prompt and present in the reference.
        - No omissions of critical facts.
        - All requested attributes (e.g., age AND email, not just one) must be present.
        - If the reference provides a multi-part answer, all parts must be covered.""
    },
    rating_rubric={
        "5": "Excellent: The response is perfectly complete. All key information requested by the prompt and present in the reference is included.** There are absolutely no omissions.",
        "3": "Good: The response is mostly complete. It has only a slight, non-critical omission that does not impact the core understanding or utility of the answer.",
        "1": "Poor: The response is critically incomplete. Essential parts of the requested information are missing, making the answer less useful or unusable for the user's purpose."
    },
    input_variables=["prompt", "reference", "response"],
),
)

tool_use_metric = TrajectorySingleToolUse(tool_name="list_table_ids")

```

我也納入了 TrajectorySingleToolUse 指標，用於[軌跡評估](#)。如果這些指標存在，評估回應中就會納入代理程式工具呼叫 (包括代理程式產生並針對 BigQuery 執行的原始 SQL)，方便您詳細檢查。

TrajectorySingleToolUse 指標會判斷代理程式是否使用特定工具。在本例中，我選擇 list_table_ids，因為我們預期這個工具會針對評估資料集中的每個問題呼叫。與其他軌跡指標不同，這項指標不需要您為評估資料集中的每個問題指定所有預期的工具呼叫和引數。

10. 建立評估工作

`EvalTask` 會採用評估資料集和自訂指標，並設定新的評估實驗。

這個函式 `run_eval` 是評估作業的主要引擎。這個函式會逐一處理 `EvalTask`，針對資料集中的每個問題執行代理程式。系統會記錄代理程式對每個問題的回覆，然後使用您先前定義的指標進行評分。

在 `evaluate_agent.py` 檔案底部複製/貼上下列程式碼：

```
def run_eval():
    eval_dataset = pd.read_json("evaluation_dataset.json")

    # Generate a unique run name
    current_time = datetime.now().strftime("%Y%m%d-%H%M%S")
    experiment_run_id = f"{current_time}-{uuid.uuid4().hex[:8]}"

    print(f"--- Starting evaluation: ({experiment_run_id}) ---")

    # Define the evaluation task with your dataset and metrics
    eval_task = EvalTask(
        dataset=eval_dataset,
        metrics=[
            factual_accuracy_metric,
            completeness_metric,
            tool_use_metric,
        ],
        experiment="evaluate-bq-data-agent"
    )

    try:
        eval_result = eval_task.evaluate(
            runnable=get_agent_response, experiment_run_name=experiment_run_id
        )
        save_evaluation_results(eval_result, experiment_run_id)
        print_evaluation_summary(eval_result)

    except Exception as e:
        print(f"An error occurred during evaluation run: {e}")

if __name__ == "__main__":
    run_eval()
```

結果會經過摘要處理，並儲存至 JSON 檔案。

11. 執行評估作業

現在您已準備好代理程式、評估指標和評估資料集，可以執行評估作業。

返回 Cloud Shell，確認您位於 `bigquery-adk-codelab` 目錄中，然後使用下列指令執行評估指令碼：

```
python evaluate_agent.py
```

評估作業進行時，畫面會顯示類似以下的輸出內容：

```
Evaluation Took:11.410560518999773 seconds
Results for run '20250922-130011-300ea89b' saved to
eval_results/bq_agent_eval_results_20250922-130011-300ea89b.json
- Row Count: row_count: 15.00
- Factual Accuracy Metric: factual_accuracy_metric/mean: 2.60
- Factual Accuracy Metric/Std: factual_accuracy_metric/std: 1.88
- Completeness Metric: completeness_metric/mean: 2.87
- Completeness Metric/Std: completeness_metric/std: 2.07
- Trajectory Single Tool Use: trajectory_single_tool_use/mean: 0.90
- Trajectory Single Tool Use/Std: trajectory_single_tool_use/std: 0.32
- Latency In Seconds: latency_in_seconds/mean: 9.77
- Latency In Seconds/Std: latency_in_seconds/std: 5.65
- Failure: failure/mean: 0.00
- Failure/Std: failure/std: 0.00
```

如果遇到下列任何錯誤，表示代理程式在特定執行期間未呼叫任何工具；您可以在下一個步驟中進一步檢查代理程式行為。

```
Error encountered for metric trajectory_single_tool_use at dataset index 1: Error:
400 List of Field:
trajectory_single_tool_use_input.instances[0].predicted_trajectory; Message: Required
field is not set.
```

解讀結果：

前往 `data_agent_app` 目錄中的 `eval_results` 資料夾，然後開啟名為 `bq_agent_eval_results_*.json` 的評估結果檔案：

```

"summary_metrics": {
  "row_count": 15,
  "factual_accuracy_metric/mean": 2.6,
  "factual_accuracy_metric/std": 1.8822478962286406,
  "completeness_metric/mean": 2.8666666666666667,
  "completeness_metric/std": 2.065591117977289,
  "trajectory_single_tool_use/mean": 0.9,
  "trajectory_single_tool_use/std": 0.31622776601683794,
  "latency_in_seconds/mean": 9.768761846866498,
  "latency_in_seconds/std": 5.654001879067884,
  "failure/mean": 0.0,
  "failure/std": 0.0
},
"pointwise_metrics": [
  {
    "prompt": "What tables are available in the dataset `ecommerce`?",
    "reference": "The tables available in the dataset `ecommerce` are: `distribution_centers`, `events`, `inventory_items`, `",
    "response": "The `ecommerce` dataset contains the following tables: `distribution_centers`, `events`, `inventory_items`, `",
    "latency_in_seconds": 4.95319563799967,
    "failure": 0,
    "predicted_trajectory": [
      {
        "tool_name": "list_table_ids",
        "tool_input": {
          "dataset_id": "ecommerce",
          "project_id": "bq-adk-demo"
        }
      }
    ],
    "factual_accuracy_metric/explanation": "All factual information in the AI's response precisely matches the reference. The",
    "factual_accuracy_metric/score": 5.0,
    "completeness_metric/explanation": "The AI's response perfectly lists all the tables available in the `ecommerce` dataset.",
    "completeness_metric/score": 5.0,
    "trajectory_single_tool_use/score": 1.0
  },

```

- **摘要指標**：提供代理程式在資料集中的整體成效。
- **逐點事實準確度和完整度指標**：分數越接近 5，代表準確度和完整度越高。每道問題都會有分數，並附上說明，解釋獲得該分數的原因。
- **預測軌跡**：這是代理用來產生最終回覆的工具呼叫清單。這樣我們就能查看代理程式產生的任何 SQL 查詢。

從上表可看出，平均完整度和事實準確度的平均分數分別為 2.87 和 2.6。

結果不太理想！讓我們試著提升服務專員的問答能力。

12. 提升代理的評估結果

前往 `bigquery-adk-codelab` 目錄中的 [agent.py](#)，然後更新 Agent 的模型和系統指令。請記得將 `<YOUR_PROJECT_ID>` 替換為您的專案 ID：

```
root_agent = Agent(
    model="gemini-2.5-flash",
    name="bigquery_agent",
    description="Agent that answers questions about BigQuery data by executing SQL queries.",
    instruction=(
        """
        You are a data analysis agent with access to several BigQuery tools.
        Use the appropriate tools to fetch relevant BigQuery metadata and execute SQL queries.
        You must use these tools to answer the user's questions.
        Run these queries in the project-id: '<YOUR_PROJECT_ID>' on the `ecommerce` dataset.
        """
    ),
    tools=[bigquery_toolset]
)
```

現在返回終端機，重新執行評估：

```
python evaluate_agent.py
```

您應該會發現結果現在好多了：

```
=====

--- Aggregated Evaluation Summary ---
Total questions in evaluation dataset: 15
Average Completeness Score: 4.73
Average Factual Accuracy Score: 4.20

=====
```

評估代理程式是反覆的過程。如要進一步提升評估結果，您可以調整系統指令、模型參數，甚至是 BigQuery 中的中繼資料 - 請參閱這些[提示和訣竅](#)，取得更多想法。

13. 清除

為避免系統持續向您的 Google Cloud 帳戶收費，請務必刪除我們在研討會期間建立的資源。

如果您為本程式碼研究室建立了任何特定 BigQuery 資料集或資料表 (例如電子商務資料集)，不妨將其刪除：

```
bq rm -r $PROJECT_ID:ecommerce
```

如要移除 bigquery-adk-codelab 目錄和當中內容，請執行下列指令：

```
cd .. # Go back to your home directory if you are still in bigquery-adk-codelab  
rm -rf bigquery-adk-codelab
```

14. 恭喜

恭喜！您已使用 Agent Development Kit (ADK) 成功建構及評估 BigQuery 代理，您現在已瞭解如何使用 BigQuery 工具設定 ADK 代理程式，以及如何使用自訂評估指標評估其效能。

參考文件

- [Agent Development Kit 說明文件](#)
 - [BigQuery 說明文件](#)
 - [Vertex AI Evaluation 說明文件](#)
 - [ADK meets BigQuery 部落格系列文章](#)
-